The loading, splitting, chunking, and embedding options provided are the only basic options supported by LangChain for now. In future versions of Nexus, more options and patterns will be offered. The code to support other options can be added directly to Nexus.

We won't cover the code that performs the RAG as it's very similar to what we already covered. Feel free to review the Nexus code, particularly the `KnowledgeManager` class in the `knowledge_manager.py` file.

While the retrieval patterns for knowledge and memory are quite similar for augmentation, the two patterns differ when it comes to populating the stores. In the next section, we'll explore what makes memory in agents unique.

8.6 Implementing memory in agentic systems

Memory in agents and AI applications is often described in the same terms as cognitive memory functions. *Cognitive* memory describes the type of memory we use to remember what we did 30 seconds ago or how tall we were 30 years ago. Computer memory is also an essential element of agent memory, but one we won't consider in this section.

Figure 8.13 shows how memory is broken down into sensory, short-term, and long-term memory. This memory can be applied to AI agents, and this list describes how each form of memory maps to agent functions:

- *Sensory memory in AI*—Functions such as RAG but with images/audio/haptic data forms. Briefly holds input data (e.g., text and images) for immediate processing but not long-term storage.
- *Short-term/working memory in AI*—Acts as an active memory buffer of conversation history. We're holding a limited amount of recent input and context for immediate analysis and response generation. Within Nexus, short- and long-term conversational memory is also held in the context of the thread.
- *Long-term memory in AI*—Longer-term memory storage relevant to the agent's or user's life. Semantic memory provides a robust capacity to store and retrieve relevant global or local facts and concepts.